

explain me datetime module for data engineering

Absolutely. For **Data Engineering**, Python's `datetime` module is important because you constantly work with:

- Event timestamps
- ETL/ELT pipelines
- Data partitioning
- Incremental loads
- Data quality checks
- Logs
- Batch windows
- Date-based filtering
- Time differences
- Scheduling

Let's learn it **from zero**, but focus only on what a Data Engineer actually needs.

1. What is `datetime` ?

Python's built-in `datetime` module provides tools for working with **dates and times**.

</> Python



Run

The main objects you need to know are:

```
date
time
datetime
timedelta
timezone
```



The most important one for Data Engineering is:

</> Python



Run

2. date

A `date` represents a calendar date:

```
2026-09-07
```



It contains:

```
year  
month  
day
```



Example:

```
from datetime import date  
  
d = date(2026, 9, 7)
```

</> Python



▶ Run

Output:

```
2026-09-07
```



You can access individual components:

```
print(d.year)
```

</> Python



▶ Run

Output:

```
2026  
9  
7
```



Data Engineering use

Dates are commonly used for:

```
partitioning  
daily reports  
incremental loads  
data retention  
date filtering
```



For example:

```
event_date = 2026-09-07
```



could become a partition:

```
/year=2026/month=09/day=07/
```



3. datetime

`datetime` represents both **date** + **time**.

For example:

```
2026-09-07 04:05:30
```



It contains:

```
year  
month  
day  
hour  
minute  
second  
microsecond
```



Example:

```
from datetime import datetime  
  
dt = datetime(2026, 9, 7, 4, 5, 30)
```

</> Python



Run

Output:

2026-09-07 04:05:30



Access components:

```
dt.year  
dt.month  
dt.day  
dt.hour
```

</> Python



Run

4. Getting the current date/time

This is extremely common in pipelines.

Current datetime

```
from datetime import datetime  
  
now = datetime.now()
```

</> Python



Run

Example:

2026-09-07 04:05:30.123456



Current date

```
from datetime import date
```

```
today = date.today()
```

</> Python



Run

Output:

2026-09-07



5. datetime.now() vs datetime.today()

You'll commonly see:

</> Python



Run

and:

</> Python



Run

Both return the current local datetime.

For Data Engineering, you'll more commonly see:

</> Python



Run

Upgrade



6. Creating a datetime manually

The structure is:

</> Python



Run

```
datetime(year, month, day, hour, minute, second)
```

Example:

</> Python



Run

```
from datetime import datetime

dt = datetime(
    2026,
    9,
    7,
    14,
    30,
    0
)

print(dt)
```

Output:

2026-09-07 14:30:00



7. Parsing strings into datetime

This is **VERY** important for Data Engineering.

Real data often arrives as strings:

2026-09-07 14:30:00



Python needs to convert that string into a real datetime object.

Use:

</> Python



Run

```
datetime.strptime()
```

Example:

</> Python



Run

```
from datetime import datetime

dt = datetime.strptime(
    "2026-09-07 14:30:00",
```

```
"%Y-%m-%d %H:%M:%S"  
)  
  
print(dt)
```

Now Python understands it as a datetime.

8. Understanding `strftime` and `strptime`

These two are extremely important.

Think:

```
strptime = String → datetime  
strftime = datetime → String
```



`strptime`

```
"2026-09-07 14:30:00"  
↓  
datetime
```



`strftime`

```
datetime  
↓  
"2026-09-07 14:30:00"
```



9. `strptime()` format codes

You need to understand the common format codes.

Code	Meaning
%Y	4-digit year
%y	2-digit year
%m	month

Code	Meaning
%d	day
%H	hour, 24-hour
%I	hour, 12-hour
%M	minute
%S	second
%f	microsecond
%p	AM/PM

For:

2026-09-07 14:30:45

Copy

the format is:

</> Python

Copy

Run

```
"%Y-%m-%d %H:%M:%S"
```

10. Different input formats

Data doesn't always come in the same format.

For example:

2026-09-07

Copy

</> Python

Copy

Run

```
datetime.strptime(  
    "2026-09-07",  
    "%Y-%m-%d"  
)
```

Another:

07/09/2026



</> Python



Run

```
datetime.strptime(  
    "07/09/2026",  
    "%d/%m/%Y"  
)
```

Another:

09-07-2026



</> Python



Run

```
datetime.strptime(  
    "09-07-2026",  
    "%m-%d-%Y"  
)
```

This matters a lot when ingesting data from different systems.

11. Formatting datetime into strings

Suppose:

</> Python



Run

```
from datetime import datetime  
  
dt = datetime(2026, 9, 7, 14, 30, 45)
```

You want:

2026-09-07



Use:

</> Python



Run

```
dt.strftime("%Y-%m-%d")
```

You can also produce:

```
2026/09/07
```



with:

</> Python



Run

```
dt.strftime("%Y/%m/%d")
```

Or:

```
07-09-2026
```



with:

</> Python



Run

```
dt.strftime("%d-%m-%Y")
```

12. Extracting date/time components

Given:

</> Python



Run

```
dt = datetime(2026, 9, 7, 14, 30, 45)
```

you can extract:

</> Python



Run

```
dt.year  
dt.month  
dt.day  
dt.hour
```

```
dt.minute  
dt.second
```

You can also get:

```
</> Python
```



▶ Run

```
dt.date()
```

which gives:

```
2026-09-07
```



And:

```
</> Python
```



▶ Run

```
dt.time()
```

which gives:

```
14:30:45
```



13. timedelta

This is another **very important** concept.

`timedelta` represents a **duration**.

For example:

```
5 days  
3 hours  
20 minutes
```



Import:

```
</> Python
```



▶ Run

```
from datetime import timedelta
```

You can create one:

</> Python



▶ Run

```
delta = timedelta(days=5)

print(delta)
```

Output:

5 days, 0:00:00



14. Adding time

Suppose:

</> Python



▶ Run

```
from datetime import datetime, timedelta

dt = datetime(2026, 9, 7)
```

Add 5 days:

</> Python



▶ Run

```
new_date = dt + timedelta(days=5)

print(new_date)
```

Result:

2026-09-12



You can also add:

</> Python



▶ Run

```
timedelta(
    days=2,
    hours=5,
```

```
minutes=30  
)
```

15. Subtracting time

</> Python



Run

```
dt - timedelta(days=7)
```

This is extremely useful in Data Engineering.

For example:

</> Python



Run

```
yesterday = datetime.now() - timedelta(days=1)
```

Or:

</> Python



Run

```
last_week = datetime.now() - timedelta(days=7)
```

Or:

</> Python



Run

```
last_hour = datetime.now() - timedelta(hours=1)
```

16. Difference between two dates

Suppose:

</> Python



Run

```
start = datetime(2026, 9, 1)  
end = datetime(2026, 9, 7)
```

You can subtract them:

</> Python



Run

```
difference = end - start

print(difference)
```

Result:

6 days, 0:00:00



You can get:

</> Python



Run

```
difference.days
```

Result:

6



This is useful for things like:

```
pipeline duration
processing time
customer age
data freshness
SLA checks
```



17. Comparing datetimes

You can compare datetime objects directly.

</> Python



Run

```
date1 = datetime(2026, 9, 1)
date2 = datetime(2026, 9, 7)
```

Then:

</> Python



Run

```
date1 < date2
```

returns:

```
True
```

You can use:

```
</> Python
```

```
>  
<  
>=  
<=  
==  
!=
```

Run

This becomes useful for filtering records.

Conceptually:

```
Give me records where event_time > last_processed_time
```

That's a very common Data Engineering operation.

18. A VERY important Data Engineering concept: timestamps

You'll frequently see data like:

```
2026-09-07 14:30:45
```

This is a **timestamp**.

For example, a table might contain:

OrderID	CustomerID	OrderDate
1001	25	2026-09-07 10:15:30
1002	31	2026-09-07 11:20:15

Python can represent those timestamps using:

</> Python



Run

`datetime`

19. UTC and time zones

This is one of the most important topics for real Data Engineering.

Suppose your server is in:

Egypt



but your data source is in:

USA



and another system is in:

Europe



You don't want pipelines interpreting timestamps differently.

That's why **UTC** is commonly used.

Conceptually:

Local Time



UTC



Store/process



Convert to local time when needed



20. Naive vs timezone-aware datetime

A datetime without timezone information is called **naive**.

Example:

</> Python



Run

```
datetime.now()
```

It tells you the time but doesn't explicitly tell you:

Which timezone?



A timezone-aware datetime contains timezone information.

Modern Python commonly uses:

</> Python



Run

```
from datetime import datetime, timezone

now_utc = datetime.now(timezone.utc)

print(now_utc)
```

You'll see something like:

2026-09-07 01:05:30+00:00



The:

+00:00



indicates UTC.

For production Data Engineering, **timezone-aware timestamps are much safer** than blindly using local time.

21. ISO 8601

You'll encounter this format constantly in APIs, logs, cloud systems, and data pipelines:

2026-09-07T14:30:45+00:00



This is an **ISO 8601 timestamp**.

Python can work with it using:

</> Python



Run

```
datetime.fromisoformat()
```

Example:

</> Python



Run

```
dt = datetime.fromisoformat(  
    "2026-09-07T14:30:45+00:00"  
)
```

And you can produce ISO format with:

</> Python



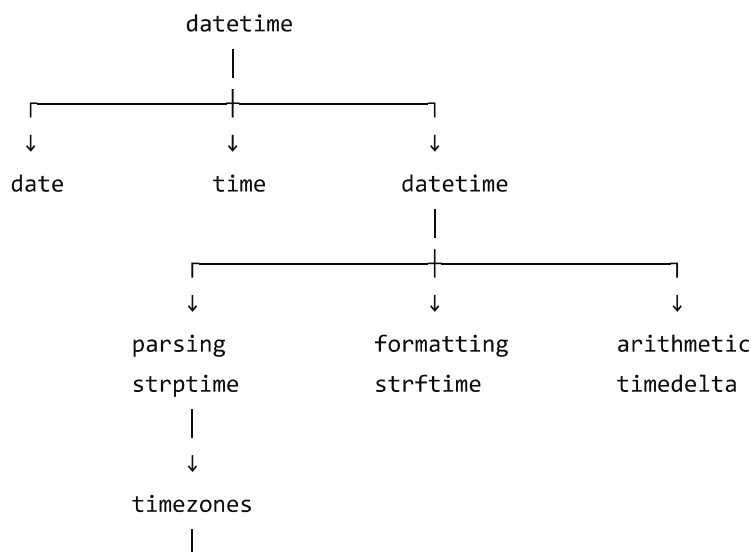
Run

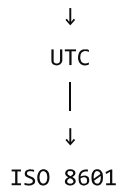
```
dt.isoformat()
```

This is particularly useful when exchanging timestamps between systems.

22. datetime concepts you should master for Data Engineering

I recommend learning them in this order:





23. What you actually need to know as a Data Engineer

Don't try to memorize every function in the module.

Focus heavily on these:

Level 1 — Must know

</> Python



▶ Run

```
from datetime import datetime, date, timedelta
```

</> Python



▶ Run

```
datetime.now()  
date.today()
```

</> Python



▶ Run

```
datetime.strptime()  
datetime.strftime()
```

</> Python



▶ Run

```
timedelta()
```

</> Python



▶ Run

```
datetime.fromisoformat()  
datetime.isoformat()
```

And:

</> Python



▶ Run

```
dt.year  
dt.month  
dt.day  
dt.hour  
dt.minute  
dt.second
```

Level 2 — Very important

Understand:

```
UTC  
timezone-aware datetime  
naive datetime  
ISO 8601
```



Level 3 — Useful

Understand:

```
date arithmetic  
datetime comparisons  
time differences  
date extraction  
timestamp normalization
```



24. Why this matters in your Data Engineering path

You'll use these concepts when working with:

SQL

```
</> SQL  
  
WHERE OrderDate >= ...
```



Pandas

```
</> Python  
  
pd.to_datetime(...)
```



▶ Run

PySpark

</> Python



▶ Run

```
to_timestamp(...)
```

</> Python



▶ Run

```
to_date(...)
```

Streaming

```
event_time  
processing_time  
watermark  
window
```



ETL

```
last successful load  
current load time  
incremental data
```



Data Warehouses

```
created_at  
updated_at  
load_date  
effective_date  
expiry_date
```



Partitioning

```
year=2026  
month=09  
day=07
```



So learning Python `datetime` isn't just about Python syntax. It's the foundation for understanding **time in data pipelines**.

Your next step

I recommend we learn this practically in this order:

1. datetime **basics** → 2. strptime/strftime → 3. timedelta → 4. UTC/timezones → 5. timestamps & ISO 8601 → 6. Data Engineering exercises → 7. datetime with Pandas → 8. datetime with PySpark.

That will take you from basic Python dates to the kind of timestamp handling you'll encounter in real ETL/streaming pipelines.



ChatGPT can make mistakes. Check important info.